

[Dashboard](#) / [My courses](#) / [Computer Engineering & IT](#) / [CSE-odd-sem-23-24](#) / [DSA-I -odd-sem-23-24](#) / [ESE 2023](#) / [Slot 1](#)

Started on	Wednesday, 29 November 2023, 10:01 AM
State	Finished
Completed on	Wednesday, 29 November 2023, 1:01 PM
Time taken	3 hours
Grade	55.50 out of 60.00 (92.5%)

Question 1

Complete

Mark 0.50 out of 1.00

Which of the following code pieces will result in a segmentation fault. Select one or more:

☐ a.

```
int *p, *q, a[16], i, j;
i = 10;
p = &a[5];
q = a - 1 + i;
j = q - p + 3;
q = p + j;
*q = 20;
```

☒ b.

```
int main()
{
    int *p;
    printf("%d", *p);
    return 0;
}
```

☒ c.

```
int main()
{
    int *p;
    printf("%d", *p);
    return 0;
}
```

☐ d.

```
int main()
{
    char *str;
    str = "GfG";
    *(str+1) = 'n';
    return 0;
}
```

The correct answers are:

```
int main()
```

```
{
    int *p;
    printf("%d", *p);
    return 0;
}, int main()
{
    int *p;
    printf("%d", *p);
    return 0;
```

```
}  
,int main()  
{  
    char *str;  
    str = "GfG";  
    *(str+1) = 'n';  
    return 0;  
}
```

Question 2

Complete

Mark 1.00 out of 1.00

Write the output of the below code:

```
#include<stdio.h>  
int main()  
{  
    struct demo  
    {  
        char name[] = "Programming";  
        int no_of_pages = 200;  
    };  
    struct demo *p;  
    printf("%d ", p->no_of_pages);  
    printf("%s", p->name);  
    getchar();  
    return 0;  
}
```

Answer: Error

The correct answer is: Compiler Error

Question 3

Complete

Mark 1.00 out of 1.00

For the following statement to be valid in C, which option should be selected.

```
string p = "HELLO";
```

- ☐ a. typedef char[] string;
- ☐ b. typedef char* string;
- ☐ c. Such statement cannot be written in C
- ☒ d. Both A and B

The correct answer is: Both A and B

Question 4

Complete

Mark 1.00 out of 2.00

Select the most appropriate pair for the C statement on left hand side -

1] `m = malloc(5); m = NULL;`

Uninitialized pointer

2] `char*p; *p='a';`

Using Dangling Pointers

3] `scanf("%d",i);`

Segmentation Fault

4] `typedef struct node {``int val;``struct node *next;``}node;``int main() {``node *p;``node a;``p = &a;``}`

Using Dangling Pointers

The correct answer is: 1] `m = malloc(5); m = NULL;` → Lost memory, 2] `char*p; *p='a';` → Uninitialized pointer, 3] `scanf("%d",i);` → Segmentation Fault, 4] `typedef struct node {`

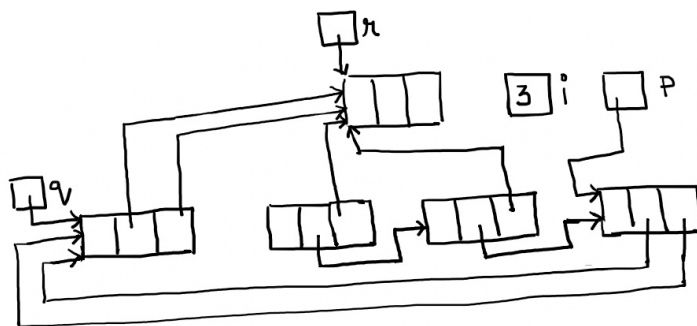
`int val;``struct node *next;``}node;``int main() {``node *p;``node a;``p = &a;``}` → Using Dangling Pointers

Question 5

Complete

Mark 2.00 out of 2.00

Which of the following piece of code will generate the below diagram?



- ☐ a.

```
struct test {
    char data;
    struct test *one;
    struct test *two;
};
typedef struct test test;
int main() {
    test *p, *q, *r;
    int i, j;
    /* assume that all mallocs succeed */
    r = (test *)malloc(sizeof(test));
    p = (test *)malloc(sizeof(test));
    q = p;
    for(i = 0; i < 4; i++) {
        p->two = r;
        r->two = p;
        p = p->one;
        p->one = (test *)malloc(sizeof(test));
    }
    p->one = p->two = q;
}
```
- ☒ b.

```
struct test {
    char data;
    struct test *one;
    struct test *two;
};
typedef struct test test;
int main() {
    test *p, *q, *r;
    int i, j;
    /* assume that all mallocs succeed */
    p = (test *)malloc(sizeof(test));
    r = (test *)malloc(sizeof(test));
    q = p;
    for(i = 0; i < 3; i++) {
        p->two = r;
        r->two = p;
        p->one = (test *)malloc(sizeof(test));
        p = p->one;
    }
    p->one = p->two = q;
    q->one = q->two = r;
}
```
- ☐ c.

```
struct test {
    char data;
    struct test *one;
    struct test *two;
};
typedef struct test test;
```

```
struct test {  
  
    char data;  
    struct test *one;  
    struct test *two;  
};  
typedef struct test test;  
int main() {  
    test *p, *q, *r;  
    int i, j;  
    /* assume that all mallocs succeed */  
    p = (test *)malloc(sizeof(test));  
    r = (test *)malloc(sizeof(test));  
    q = p;  
    for(i = 0; i < 3; i++) {  
        p->two = r;  
        r->two = p;  
        p->one = (test *)malloc(sizeof(test));  
        p = p->one;  
    }  
    p->one = p->two = q;  
    q->one = q->two = r;  
}
```

The correct answer is:

Question 6

Complete

Mark 3.00 out of 3.00

Write a C function **BinarySearch()** that takes array of integer, its size and a integer to be searched as parameters and returns 1 if the number is found and zero if not found. Perform the searching operation using Binary search method.

Note: Name of function should be exactly same as given. Template for main() is given complete it.

Answer: (penalty regime: 10, 20, ... %)

[Reset answer](#)

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <limits.h>
5
6
7
8 void sort(int arr[],int n){
9     if (n<=1){
10         return;
11     }
12     for (int i=0;i<n;i++){
13         for (int j=0;j<n;j++){
14             if (arr[j]>arr[j+1]){
15                 int temp=arr[j];
16                 arr[j]=arr[j+1];
17                 arr[j+1]=temp;
18             }
19         }
20     }
21     return;
22 }
23
24
25 int BinarySearch(int arr[],int n,int key) {
26     if (n<=0){
27         return 0;
28     }
29     sort(arr,n);
30     int start=0;
31     int end=n-1;
32     while (start<=end){
33         int mid=start+(end-start)/2;
34         if (arr[mid]==key){
35             return 1;
36         }
37         else if(arr[mid]<key){
```

Passed all tests! ✓

Question author's solution (C):

```
1 #include <stdio.h>
2
3 int BinarySearch(int array[], int x, int t) {
4     int low = 0;
5     int high = x-1;
6     // Repeat until the pointers low and high meet each other
7     while (low <= high) {
8         int mid = low + (high - low) / 2;
9         if (array[mid] == t)
10             return 1;
11         if (array[mid] < t)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16     return 0;
17 }
```

CorrectMarks for this submission: 3.00/3.00. Accounting for previous tries, this gives **0.00/3.00**.Question **7**

Complete

Mark 1.00 out of 1.00

Given a [queue](#) with a linked list implementation, the rear pointer points to the rear node of the [queue](#) and the front pointer points to the front node of the [queue](#). Which of the following operations is impossible to do in $O(1)$ time?

- ☐ a. Delete the front item from the list.
- ☒ b. Delete the rear from the list.
- ☐ c. insert at the front of the list.
- ☐ d. None

The correct answer is: Delete the rear from the list.

Question **8**

Complete

Mark 1.00 out of 1.00

State whether True or False

A character pointer can be initialized with NULL or 0.

- ☒ a. Yes
- ☐ b. No

The correct answer is: Yes

Question **9**

Complete

Mark 2.00 out of 2.00

An array of size 9 is given. Three stacks are implemented using this array in order 1,2 and 3. Size of each stack is :

Stack 1 - size 2

Stack 2 - size 3

Stack 3 - size 4

Initially array contains no elements.

Operation push(1, 2) means push value 2 in stack number 1.

Assume all push operations happen from right of stack. After performing the following operations, what are the contents of the array starting from index 0?

push(1, 2)

push(2, 3)

push(3, 4)

push(1, 5)

push(2, pop(1))

push(2, 6)

push(3, 8)

push(3, 7)

push(3, pop(1))

push(1, pop(2))

push(1, 5)

push(2, 1)

Note: Write your answer where every value is separated by space

Eg: 1 2 3 4 5

Answer: 6 5 3 5 1 4 8 7 2

The correct answer is: 6 5 3 5 1 4 8 7 2

Question **10**

Complete

Mark 1.00 out of 1.00

If circular linked list is used to implement a [queue](#), how can we implement enqueue and dequeue operation using single pointer variable in constant time?

- ☒ a. Make that pointer point to last node
- ☐ b. Make that pointer point to any node
- ☐ c. Not possible with single pointer
- ☐ d. Make that pointer point to first node

The correct answer is: Make that pointer point to last node

Question 11

Complete

Mark 0.00 out of 1.00

Which of the following application makes use of a circular linked list?

- ☐ a. Undo operation in a text editor.
- ☒ b. Recursive function calls.
- ☐ c. Allocating CPU to resources.
- ☐ d. Implement Hash Tables.

The correct answer is:

Allocating CPU to resources.

Question 12

Complete

Mark 1.00 out of 1.00

In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is

Answer:

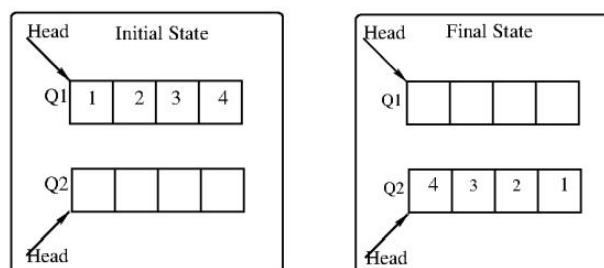
The correct answer is: n

Question 13

Complete

Mark 0.00 out of 2.00

Consider the queues Q1 containing four elements and Q2 containing none (shown as the Initial State in the figure). The only operations allowed on these two queues are Enqueue(Q, element) and Dequeue(Q). The minimum number of Enqueue operations on Q1 required to place the elements of Q1 in Q2 in reverse order (shown as the Final State in the figure) without using any additional storage is



The correct answer is: 0

Question **14**

Complete

Mark 2.00 out of 2.00

Write a C function to count the number of elements in a doubly linked list.

Note: Write only count() function. Include structure of doubly linked list.

```
typedef struct node{
    struct node* next;
    struct node* prev;
    int data;
}node;

typedef struct list{
    struct node* front;
    struct node* tail;
}list;

int count(struct list head){
    if (head->front==NULL && head->tail==NULL){
        return 0;
    }
    struct node* curr=head->front;
    int i=0;
    do{
        i++;
        curr=curr->next;
    }while (curr != head->front);

    return i;
}
```

Question **15**

Complete

Mark 5.00 out of 5.00

Write following [functions](#) with suitable prototypes for ADT Circular Linked List :

`init_CLL()` **(0.5 mark)** // initializes circular linked list of integers

`append()` **(1 mark)** // to add an element in the end of the list.

`traverse()` **(0.5 mark)** // to display all the elements of the list starting from first element to last.

`del_odd()` **(3 marks)** /*This function deletes all odd integers in the list .

For example, if the list is: {1, 2, 8, 9,12,16,18,11,14,13}

after the function call the resultant circular list should be:

{2, 8,12,16,18,14} */

You are free to include more [functions](#).

Skeleton of function `main()` is given below, use same by replacing commented statements by actual function calls:

```
int main() {  
    CLL L1;  
    //call init() // call init for list  
    // call append() multiple times to insert elements in the example.  
    // call traverse()  
    // call del_odd()  
    // call traverse()  
    return 0;  
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct node{
int data;
struct node* next;
}node;

typedef struct list{
struct node* front;
struct node* tail;
int count;
}list;

void init(struct list* l){
l->front=NULL;
l->tail=NULL;
l->count=0;
return;
}

int isEmpty(struct list l){
return l.count==0;
}

void append(struct list* l,int data){
struct node* temp=(struct node*)malloc(sizeof(struct node));
if (temp){
temp->data=data;
temp->next=NULL;
}
else{
return;
}

if (isEmpty(*l)){
l->front=temp;
l->tail=temp;
}
```

Question **16**

Complete

Mark 5.00 out of 5.00

Define ADT SLL (Singly Linked List of characters).

Write following [functions](#) with suitable prototypes for ADT SLL:

init_SLL() // **(0.5 Mark)** to initialize the list

insert_beg() // **(1 Mark)** to add add an element at the beginning of the list.

traverse() // **(0.5 Marks)** to display all the elements of the list starting from last element to first.

is_palindrome() //(3 marks)

/* The [functions](#) determines the given list is a palindrome or not. For example, if the list is: {1, 2, 3, 2, 1} is a palindrome. For example, if the list is: {1, 2, 3, 1, 2, 1} is a not a palindrome.*/

You are free to include more [functions](#).

Skeleton of function main() is given below, use same by replacing commented statements by actual function calls:

```
int main() {  
    SLL L1, L2;  
    //call init() for both lists  
    // call append() multiple times to insert elements in the respective list as in the example.  
    // call traverse() both lists  
    // call is_palindrome() for both lists  
    return 0;  
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct node{
char data;
struct node* next;
}node;

void init(struct node** head){
(*head)=NULL;
return;
}

void append(struct node** head,char data){
struct node* temp=(struct node*)malloc(sizeof(struct node));
if (temp){
temp->data=data;
temp->next=NULL;
}
else{
return;
}
if ((*head)==NULL){
(*head)=temp;
return;
}
struct node* curr=(*head);
while (curr->next != NULL){
curr=curr->next;
}
curr->next=temp;
return;
}

void insert_beg(struct node** head,char data){
struct node* temp=(struct node*)malloc(sizeof(struct node));
if (temp){
temp->data=data;
```

Question **17**

Complete

Mark 5.00 out of 5.00

Write C [functions](#) to represent a sparse matrix using linked list. Perform following operation:

1. Structure for the sparse matrix
2. Initialize the sparse matrix
3. Insert elements in sparse matrix
4. Display the sparse matrix (In sparse matrix form which includes zero and non-zero elements)
5. Perform addition of two sparse matrices and store the result in a new sparse matrix.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct node{
int data;
int row;
int col;
struct node* next;
}node;

typedef struct spm{
struct node* front;
int nr;
int nc;
int count;
}spm;

void init(struct spm* st,int nr,int nc){
st->front=NULL;
st->nr=nr;
st->nc=nc;
st->count=0;
}

int isEmpty(struct spm st){
return st.count==0;
}

void append(struct spm* st,int r,int c,int v){
struct node* temp=(struct node*)malloc(sizeof(struct node));
if (temp){
temp->data=v;
temp->r=r;
temp->c=c;
}
else{
return;
}
if(isEmpty(*st)){
```


Question 18

Complete

Mark 1.00 out of 1.00

What output does the following code produce?

Assume the push, pop, enqueue, dequeue as standard operations of Stack and [Queue](#)

```
int main() {
    Stack s;
    Queue q;
    for(int i = 0; i < 5; i++){
        push(&s, i);
        enqueue(&q, i);
    }
    for(int i=0; i < 5; i++){
        push(&s, dequeue(&q));
        enqueue(&q, pop(&s));
    }
    while (!isEmpty(q))
        printf("%d ", dequeue(&q));
}
```

- ☐ a. 4 3 2 1 0 0 1 2 3 4
- ☐ b. 0 1 2 3 4 4 3 2 1 0
- ☒ c. 0 1 2 3 4
- ☐ d. 4 3 2 1 0

The correct answer is: 0 1 2 3 4

Question **19**

Complete

Mark 1.00 out of 1.00

What's the output of this code?

```
#include <stdio.h>
#include "stack.h"
/* assume that stack doesnt get full */
int main(){
    stack s;
    int i;
    init(&s);
    for(i = 0; i < 8; i++){
        if(i % 3 == 0)
            push(&s, i + 1);
        if(i % 5 == 0)
            push(&s, i - 1);
    }
    while(!isempty(&s))
        printf("%d\n", pop(&s));
    return 0;
}
```

- ☐ a. 7 4 1 1
- ☒ b. 7 4 4 -1 1
- ☐ c. 7 4
- ☐ d. 1 4 7

The correct answers are: 7 4 1 1, 7 4 4 -1 1

Question **20**

Complete

Mark 2.00 out of 2.00

Consider the following magic function psuedo code that does some operation on an integer

```
void magic (int n) {  
    stack s; // assume a empty stack created  
    while (n > 0) {  
        push(n%2, &s);  
        n = n/2;  
    }  
    while (!empty(s))  
        printf("%d", pop(&s)); // pop an element from s and  
print it  
}
```

What does magic(17) print?

NOTE: Write your answer in below box without space or quotes

Answer:

The correct answer is: 10001

Question **21**

Complete

Mark 5.00 out of 5.00

Implement a stack of integers using singly linked list in C. Invoke all stack [functions](#) in main function to demonstrate the usage of the [functions](#).

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct node{
int data;
struct node* next;
}node;

typedef struct stack{
int count;
struct node* front;
struct node* tail;
}stack;

void init(struct stack* st){
st->front=NULL;
st->tail=NULL;
st->count=0;
return;
}

int isEmpty(struct stack st){
return st.count==0;
}

void display(struct stack st){{
if (isEmpty(st)){
return;
}
struct node* curr=st.front;
while (curr != NULL){
printf("%d ",curr->data;
curr=curr->next;
}
printf("\n");
return;
}

void push(struct stack* st,int data){
```

Question **22**

Complete

Mark 2.00 out of 2.00

Consider a standard Circular [Queue](#) 'q' implementation (which has the same condition for [Queue](#) Full and [Queue](#) Empty) whose size is 11 and the elements of the [queue](#) are q[0], q[1], q[2].....q[10]. The front and rear [pointers](#) are initialized to point at q[2] . In which position will the ninth element be added?

- ☒ a. q[0]
- ☐ b. q[1]
- ☐ c. q[9]
- ☐ d. q[10]

The correct answers are: q[0], q[10]

Question **23**

Complete

Mark 2.00 out of 2.00

The following piece of code implements a circular [queue](#) (of size 10) of integers using arrays in C. The array Q contains the circular [queue](#), front and rear are the indices of the elements of the circular [queue](#) at the front and rear ends respectively.

```
void insert(cqueue *q, int val) {
if((q->front==0 && q->rear==q->size-1) ||  )
return;
else{

else
q->rear++;
q->Q[q->rear]=val;
}
if(q->front== -1)
q->front=0;
}

int delete(cqueue *q) {
int k;
if(q->front== -1)
return INT_MIN;
else {
k=q->Q[q->front];
if(q->front==q->rear)
q->front=q->rear=-1;
else {

else
q->front++;
}
}
return k;
}
```

Complete the missing snippets of code, from the options given below, so that the insert() and delete() [functions](#) are able to insert and delete an element to/from the circular [queue](#) respectively.

The correct answer is:

The following piece of code implements a circular [queue](#) (of size 10) of integers using arrays in C. The array Q contains the circular [queue](#), front and rear are the indices of the elements of the circular [queue](#) at the front and rear ends respectively.

```
void insert(cqueue *q, int val) {
if((q->front==0 && q->rear==q->size-1) || [q->rear+1==q->front])
return;
else{
[ if(q->rear==q->size-1) q->rear=0;]
else
q->rear++;
q->Q[q->rear]=val;
}
```

```
}  
if(q->front==-1)  
q->front=0;  
}  
  
int delete(cqueue *q) {  
    int k;  
    if(q->front==-1)  
        return INT_MIN;  
    else {  
        k=q->Q[q->front];  
        if(q->front==q->rear)  
            q->front=q->rear=-1;  
        else {  
            [ if(q->front==q->size-1) q->front=0;]  
            else  
                q->front++;  
        }  
    }  
    return k;  
}
```

Complete the missing snippets of code, from the options given below, so that the insert() and delete() [functions](#) are able to insert and delete an element to/from the circular [queue](#) respectively.

Question **24**

Complete

Mark 5.00 out of 5.00

Consider the below definition for circular [queue](#):

```
typedef struct CQueue
{
    int size, front, rear, len;
    int *Q;
} CQueue;
```

Write C [functions](#) :

1. to initialize the [queue](#)
2. add element in [queue](#)
3. delete element from [queue](#)
4. display the contents of [queue](#)

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct cqueue{
int size;
int front;
int rear;
int len;
int* Q;
}cqueue;

void init(struct cqueue* q,int size){
q->Q=(int *)malloc(sizeof(int)*size);
q->size=size;
q->front=0;
q->rear=0;
q->len=0;
return;
}

int isFull(struct cqueue q){
return q.len==q.size;
}

void add(struct cqueue* q,int data){
if (isFull(*q)){
return;
}
q->Q[q->len]=data;
q->len=(q->len+1)%q->size;
q->rear=data;
return;
}

int isEmpty(struct cqueue q){
return q.len==0;
}

void delete(struct cqueue* q){
```


Question **25**

Complete

Mark 1.00 out of 1.00

What is the postfix expression for the corresponding infix expression?

$(A + B^D) / (E - F) + G$

- ☒ a. $ABD^+ EF - / G+$
- ☐ b. $ABD + ^EF - / G+$
- ☐ c. $ABD + ^EF / - G+$
- ☐ d. $ABD^+ EF / - G+$

The correct answer is: $ABD^+ EF - / G+$

Question **26**

Complete

Mark 5.00 out of 5.00

Write following [functions](#) with suitable prototypes for Array :

init() (0.5 mark) // initializes an array of 20 integers

append() (1 mark) // to add an element in the array.

traverse() (0.5 mark) // to display all the elements of the array starting from first element to last.

mov_neg() (3 marks) /*This function moves all negative numbers to beginning and positive to end of the array

For example,

Input: -12, 11, -13, -5, 6, -7, 5, -3, -6

Output: -12 -13 -5 -7 -3 -6 11 6 5

*/

You are free to include more [functions](#).

Skeleton of function main() is given below, use same by replacing commented statements by actual function calls:

```
int main() {  
    CLL L1;  
    //call init() // call init for list  
    // call append() multiple times to insert elements in the example.  
    // call traverse()  
    // call mov_neg()  
    // call traverse()  
    return 0;  
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

typedef struct array{
int x;
int y;
int** arr;
int count;
int size;
}array;

void init(struct array* a,int x,int y){
a->arr=(int **)malloc(sizeof(int)*x);
for (int i=0;i<x;i++){
a->arr[i]=(int *)malloc(sizeof(int)*y);
}
for (int i=0;i<x;i++){
for (int j=0;j<y;j++){
a->arr[i][j]=0;
}
a->size=(x*y);
a->count=0;
}

int isFull(struct array a){
return a.size==a.count;
}

void append(struct array* a,int data,int x,int y){
if (isFull(*a)){
return;
}
a->arr[x][y]=data;
return;
}

int isEmpty(struct array a){
return a.count==0;
}
```

[◀ Assignment 8](#)[Lab Work ▶](#)